DIM0501 — Boas Práticas de Programação

Relatório do Projeto da Unidade 1 — AgendaFácil

Prof. Marcus Terra · DIMAp/UFRN · Semestre 2026.1 · Entrega: 25/10/2026

Integrantes (nome e matrícula)	Nome Sobrenome (matrícula)
Nome Sobrenome (matrícula)	
Linguagem	Python
Repositório ou arquivo entregue	`https://github.com/usuario/repositorio`

1. Diagnóstico do código-base

#	Categoria	Anomalia e onde está	Por que é um problema
1	Programação defensiva e segurança	Em `add`, entradas fora das regras são ignoradas em silêncio: os `if` aninhados de `h` e `dur` não têm `else`, e todo o corpo está dentro de um `try` com `except: pass`. Ex.: marcar às 20h não marca e não avisa.	Quem utiliza a função acredita que o atendimento foi marcado, pois nenhuma mensagem ou exceção é apresentada quando uma regra não é atendida. Isso faz com que o problema só seja percebido posteriormente, quando o paciente descobre que não possui horário, sem que exista qualquer indicação da causa. Além disso, o `except: pass` faz com que qualquer erro que venha a acontecer dentro de `add`, inclusive bugs do próprio código, seja ignorado, dificultando a identificação de onde um determinado problema pode estar acontecendo.
2	Manutenabilidade	Em `add`, os parâmetros da função não possuem nomes descritivos.	Para entender o que é cada um desses parâmetros, o desenvolvedor precisará navegar pelo código e tentar inferir com base nas verificações o que cada parâmetro significa, além de ter que ter algum domínio pequeno sobre o negócio para facilitar essa inferência.
3	Manutenabilidade	Em todas as funções presentes no sistema, ao ter que acessar alguma informação de um agendamento existente, é feita a obtenção desses dados com base na posição em que elas são armazenadas na estrutura de dados utilizada (lista).	Torna-se necessário o conhecimento da ordem aos quais os dados são armazenados para cada uma das linhas. Caso utilizassemos uma estrutura diferente como um objeto ou mesmo um dicionário, onde temos a associação entre chave e valor, onde essa chave indicaria exatamente qual informação que estariamos acessando (nome do paciente, data e etc.) e o valor sendo exatamente a informação ao qual armazenamos durante a inserção, isso tornaria muito mais fácil o processo de decodificar qual informação está sendo acessa, ou mesmo durante o processo de implementar uma funcionalidade diferente, acessar mais facilmente um valor ou alterar ele se torna muito mais simples por retirar a necessidade de ficar sempre se lembrando da ordem dos dados.
4	Adequação e lógica	Em `add`, não há nenhuma validação referente ao formato de data.	De acordo com a regra de negócio, a data deverá assumir o seguinte formato: AAAA-MM-DD. Qualquer outro formato é considerado como inválido.
5	Adequação e lógica	Em `add`, a duração não contém uma validação adequada, conforme defi-	De acordo com a regra de negócio, a duração deve ser entre 15 e 120 mi-

2. O que foi resolvido e o que ficou como dívida

#	Situação	Como foi resolvida, ou por que ficou
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		

3. Decisões de projeto

Decisão tomada	Alternativa descartada	Por que decidimos assim

4. Mudanças de comportamento

Entrada ou situação	O que acontecia antes	O que acontece agora

5. Como executar e gerar a documentação

Item	Resposta
Comando para compilar e executar	
Comando para gerar a documentação de API	
Onde a documentação gerada ficou no repositório	
Saída do programa de demonstração	(ver abaixo)

Saída do programa de demonstração:

```
(cole aqui a saída do programa)
```

6. Uso de ferramentas de IA

Pergunta	**Resposta**
Foram usadas ferramentas de IA neste trabalho?	☐ Não ☐ Sim
Em quais etapas, e para quê?	